



COPILOTTRAINING
TECHNICAL BRIEFING

GitHub Code Quality

| Turn Quality Standards Into an Automatic Merge Gate

Set the threshold and require the evidence; each eligible PR gets an enforceable merge decision.

GitHub Code Quality · GA July 20, 2026 · Developers, Repo Admins, Platform Teams



"How can a team enforce its quality bar on every pull request?"

Code Quality reports findings and coverage; a configured ruleset **blocks a merge when the PR misses the team's threshold.**

Developer

See coverage deltas and Autofix suggestions before your PR merges



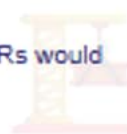
Repo Admin

Set quality and coverage thresholds in a ruleset, then test them in evaluate mode



Platform Team

Enable a pilot and inspect which PRs would be blocked



PR finding

Rules-based analysis identifies a maintainability or reliability issue

Coverage signal

CI uploads line coverage for the PR and its default-branch baseline

Merge decision

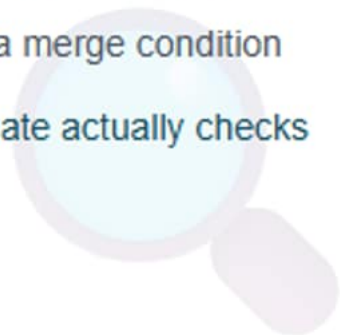
An active ruleset blocks PRs that miss the configured threshold

PART 1

Coverage-Aware Quality Gates

From coverage upload to a merge condition

See what the configured gate actually checks



PART 2

Copilot Autofix in the PR Loop

A fix proposal for a finding in the PR

Apply and verify a fix without confusing it with the gate



PART 3

Rolling Out Without Surprises

Enable → Evaluate → Active in repeatable phases

Pilot the gate, then decide where to enforce it

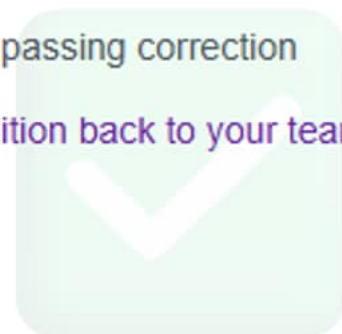


PART 4

Prove the Gate in Your Repo

Show a failing PR, then a passing correction

Take a tested merge condition back to your team



Coverage-Aware Quality Gates

Upload coverage, set a threshold, and let an active ruleset enforce it on each PR



Coverage Delta on Every PR

Cobertura XML upload shows the coverage delta on each PR



Ruleset-Enforced Threshold

Require the upload check; then a PR below the minimum cannot merge



Evaluate Before Blocking

See what would fail without blocking anyone yet

Example: PR coverage is 76%; required minimum is 80%
↳ Active quality gate: fail → merge blocked

A Coverage Threshold Can Block the Merge

Before Code Quality

CI produces a coverage report

Reviewers can inspect coverage manually

Target in a team doc

A reviewer decides when a PR falls short

The merge button does not enforce that target

After Code Quality

CI uploads Cobertura on the PR and default branch

Example: 76% vs. 80% minimum

Active coverage gate fails; merge is blocked

Author adds tests and reruns CI

Coverage meets 80%; the gate passes

76%

example PR coverage

80%

configured minimum

Blocked

until the gate passes



Require the coverage upload check: the ruleset alone does not wait for the report.

Feeding the Gate: Coverage Upload in CI

quality-gate-workflow.yml (illustrative)

```
name: Coverage
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
permissions:
  contents: read
  code-quality: write
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: pytest --cov=. --cov-report=xml
      - uses: actions/upload-code-coverage@v1
        with:
          file: coverage.xml
          language: Python
```

🔑 Required Permission

code-quality: write lets the upload step attach coverage to the PR

📄 Baseline and PR

Upload Cobertura on main and the PR; adapt test setup to your repository

🔒 Required Upload Check

Require the workflow check: the coverage rule does not wait for the upload

Two Enforcement Modes — One Field to Flip



Evaluate Mode

Reports without blocking

See how current PRs would fare — no merges stopped

Safe default for any new ruleset**Tune before committing**

Adjust threshold until false-positive rate is acceptable

Switch to active with one field change

Active Enforcement

Blocks merges on failure

Coverage upload check required; ruleset enforces the threshold

Exact violation shown to the PR author**Instant rollback available**

Switch back to evaluate with a single field change

One threshold, one rule, one check



Two Paths to the First Coverage Upload



Agent-Generated PR

Optional onboarding path

Agent opens a reviewable PR with a least-privilege workflow

Standard upload steps, minimal permissions

Review and merge like any other PR

Available on github.com in public preview



Manual Workflow

Full control over build steps and test setup

Custom test matrices

Multi-language, monorepo, or non-standard coverage tooling

Always available

Manual authoring is never blocked by the agent path

Bring your existing CI coverage pipeline

Copilot Autofix in the PR Loop

FanHub #201 shows a human-accepted Code Quality fix; #199 shows the separate coverage gate



Code Quality Finding

The PR scan flagged a generic catch in Episodes.razor



Generated Changeset

Autofix proposed TryParse; the author inspected and committed it



Verify the Result

Build passed and the original finding became outdated; keep the PR draft

`github.com/MSBart2/FanHub/pull/201`

↳ Finding → suggested changeset → author accepts → checks rerun

FanHub #201: A Human Accepts the Generated Fix

Code Quality proposed a precise correction; the author committed it and checked the new head

✗ First head: 294529d

- 1 Episodes parses a selection
int.Parse plus catch (Exception)
- 2 Invalid input falls back
Unrelated exceptions also get swallowed
- 3 First-head build passes
The compiler cannot flag this behavior
- 4 Finding is actionable
Inspect the Code Quality changeset



A passing build did not expose the broad catch

✓ Autofix head: 0b33cda

- 1 Code Quality finds the catch
Generic catch clause on the changed lines
- 2 Autofix proposes TryParse
Invalid or overflowing input still falls back
- 3 Author commits suggestion
GitHub creates the Autofix commit 0b33cda
- 4 New-head build passes
Original finding is now outdated



Author-accepted fix; PR remains draft

No coverage gate on #201; FanHub #199 proves the separate merge gate

Two Independent Features, One PR Surface

Code Quality Autofix

Bundled with Code Quality

Included with enablement — no separate setup

CodeQL posts rules-based PR findings

AI-powered proposal

Apply a useful suggestion, then rerun the scan

Ruleset enforces the configured quality threshold

Copilot Code Review

Independent enablement

Request a review or configure automatic review separately

Different evidence

Copilot comments on potential issues in the PR

Code Quality does not automatically add Copilot as reviewer

A quiet review does not replace a quality gate

Rolling Out Without Surprises

Pilot the rule in evaluate mode, then activate it when the evidence supports enforcement



Enable & Observe

Enable on pilot repos; watch scores, no enforcement yet



Evaluate Mode

Rulesets in evaluate — flag without blocking anything



Active Enforcement

Activate after inspecting would-block PRs and exceptions

Pilot repository

↳ Enable → Evaluate → Active; confirm the gate blocks an example PR

The Evaluate-First Rollout Pattern

Observe the gate on real PRs before it starts blocking merges

Phase 1

Enable & Observe

Enable on pilot repos; upload main and PR coverage; require the upload check



Phase 2

Evaluate Mode

Create rulesets with enforcement: evaluate — see what would have been blocked, tune thresholds



Phase 3

TARGET

Active Enforcement

Switch on the pilot gate; expand to other repositories after checking results



Inspect would-block PRs, tune the rule, then test one failing and one passing PR

Three Governance Layers for Deliberate Enablement



Enterprise Policy

Enterprise owners set whether Code Quality is allowed before any org can enable it

Allow for all orgs

Allow for selected orgs

Block repo-level override



Org-Level Enablement

Org admins enable Code Quality per repo, within the enterprise policy

Choose pilot repos first

Add coverage workflows

Set up evaluate rulesets



Repo-Level Control

Repo admins tune thresholds and enforcement within what the org permits

Evaluate before blocking

Inspect failed checks

Review exception requests

Estimate the Pilot Cost Before Expanding



Active Committers

\$10 per active committer per month on enabled repos; each person counts once

Trailing 90-day window

Example: 50 people = \$500/month base

Check actual license usage



AI Credits

Autofix and AI detection consume usage-based credits

Depends on model usage

Estimate from pilot activity

Copilot code review is separate



Scan Compute

CodeQL scans consume GitHub Actions minutes or self-hosted capacity

Check pilot scan frequency

Use actual runner usage

Revisit before expanding

Prove the Gate in Your Repo

FanHub PR #199: watch an isolated coverage rule block, then pass, on two commits



Fail

First head: 26.7% against a 27% minimum; GitHub blocks merge



Correct

Add tests for both Home empty-quote states; rerun coverage



Pass

Corrected head: 30% in GitHub; rule passes; PR remains draft

`github.com/MSBart2/FanHub/pull/199`

↳ Open the PR history: blocked → tested → gate passes; leave it unmerged

FanHub #199: The Same PR Crosses the Gate

First head: merge blocked

4bd7844: empty-quote state has no focused tests

Six tests pass; coverage uploads successfully

GitHub says 26.7% below 27.0%

The active ruleset disables merge

Corrected head: gate passes

ab96e8b: tests cover both empty-data paths

Eight tests pass; GitHub displays 30% coverage

All requirements met

Keep the PR draft; a human still reviews the change

26.7%

blocked first head

27.0%

branch-scoped minimum

30%

GitHub result after tests

 github.com/MSBart2/FanHub/pull/199 - Absolute minimum only: this isolated base has no uploaded coverage baseline.

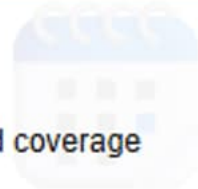
✓ WHAT YOU CAN DO TODAY

Today



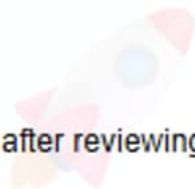
- Choose one pilot repository and a coverage threshold
- Check whether CI uploads Cobertura on main and PRs
- Inspect one rules-based PR finding

This Week



- Require the expected coverage upload check
- Create a ruleset in evaluate mode
- Record which PRs would be blocked and why

This Month



- Activate the pilot rule after reviewing results
- Test one failing and one passing PR
- Expand to another repo when the gate behaves as intended

🔑 Key Takeaway

Configure the bar, observe it in evaluate mode, then prove the active gate blocks and unblocks a PR.

 Official Documentation[GitHub Code Quality - Concepts](#)

Core concepts for PR and default-branch scanning

[Enabling GitHub Code Quality](#)

Step-by-step repo and org enablement

[Catch Issues Before Merge](#)

Ruleset-based merge gating walkthrough

[Restrict code coverage](#)

Threshold rule and required upload check

[Code Quality Billing](#)

Active-committer pricing, AI credits, and Actions minutes

 Changelog Announcements[Code Quality No Longer Adds Copilot as a Reviewer](#)

Boundary between Code Quality and Copilot review enablement

[Automatic Coverage Enablement](#)

Agent-generated coverage workflow pull request in public preview



GitHub Code Quality

TURN QUALITY STANDARDS INTO AN AUTOMATIC MERGE GATE

Automatic Gate

Coverage upload or quality finding → ruleset threshold → merge condition

Autofix in the PR

Inspect a proposed fix, apply it, and rerun the scan

Fail → Pass

A failing PR blocks; a correction that meets the bar can proceed

Enable → Evaluate → Active

Pilot the rule before it blocks merges

Which signal would your team enforce first — coverage threshold, maintainability score, or reliability score?